

Chapter 3

Visualizing Network Data

3.1 Introduction

Up until this point, we have spoken only loosely of displaying network graphs, although we have shown several examples already. Here in this chapter we consider the problem of display in its own right. Techniques for displaying network graphs are the focus of the field of *graph drawing* or *graph visualization*. Such techniques typically seek to incorporate a combination of elements from mathematics, human aesthetics, and algorithms. After a brief characterization of the elements of graph visualization in Sect. 3.2, we look at a number of ways to lay out a graph, in Sect. 3.3, followed by some ways to further decorate such layouts, in Sect. 3.4. We also look quickly at some of the unique challenges posed by the problem of visualizing large network graphs in Sect. 3.5. Finally, in Sect. 3.6, we describe options for producing more sophisticated visualizations than those currently possible using R.

3.2 Elements of Graph Visualization

Suppose we have a set of network measurements that have been encoded in a network graph representation $G = (V, E)$, and we now wish to summarize G in a visual manner. At the heart of the graph visualization problem is the challenge of creating “geometric representations of . . . combinatorial structures,” [46] using symbols (e.g., points, circles, squares, etc.) for vertices $v \in V$ and smooth curves for edges $e \in E$. For human consumption it is most convenient, of course, if a graph is drawn¹ in two-dimensional space, as opposed to three-dimensional space or on some more abstract surface. Hence, we will restrict our attention to this setting.

¹ Here and throughout we use terms like ‘draw’ only in the colloquial sense, although more formal mathematical treatments of this topic area exist (e.g., see Chap. 8 of Gross and Yellen [67]) which attach more specialized understandings to these terms.

Intuitively, it is not hard to see that there are uncountably many ways that we could lay down a candidate set of points and curves on paper to represent a graph G . The important question for any such candidate, however, is whether or not it adequately communicates the desired relational information in G . While in principle this might suggest the drawing of graphs by hand, in practice hand drawings are only realistic for very small graphs. Generally graphs of nontrivial size must be drawn, at least in part, using automated methods.

In principle, one could simply lay down the vertices in G in a random fashion (e.g., uniformly) over a given region and then draw straight lines between those vertices connected by an edge. Unfortunately, although such drawings are simple enough to create, they tend to look like little more than a ball of yarn. In order to facilitate improved automatic drawings of graphs, various specifications or requirements have evolved—some firm, and some flexible—which have been formally categorized as drawing conventions, aesthetics, and constraints. See di Battista, Eades, Tamassia, and Tollis [46] for a detailed treatment, or Kolaczyk [91, Chap. 3.4] for a brief summary.

From a practical perspective, drawing conventions, aesthetics, and constraints effectively serve to define parameters for automatic graph drawing methods, and the determination of a graph drawing frequently becomes a formal optimization over some or all of these parameters. Such optimizations typically are difficult to solve exactly in real time for graphs that are nontrivial in size. Therefore, it is common to develop computationally efficient algorithms that seek an approximate solution, often through the use of heuristics and the imposition of priorities among aesthetics.

Within **igraph**, the `plot` command calls a variety of such algorithms which, when applied to a graph object `g`, allows for the user to produce a fairly rich assortment of graph visualizations. The parameters associated with `plot` allow for both the specification of the algorithm to be used and the settings of various conventions, aesthetics, and constraints allowed for by a given algorithm.

3.3 Graph Layouts

At the heart of graph visualization is the graph layout, i.e., the placement of vertices and edges in space. There are far too many graph layout methods for us to present a full survey here. Rather, we discuss a handful of representative examples.

We will illustrate using two network graphs—a $5 \times 5 \times 5$ lattice

```
#3.1 1 > library(sand)
      2 > g.l <- graph.lattice(c(5, 5, 5))
```

and a network of ‘web-logs’ or simply ‘blogs’.

```
#3.2 1 > data(aidsblog)
      2 > summary(aidsblog)
      3 IGRAPH D--- 146 187 --
```

The blog network is a snapshot of the pattern of citation among 146 unique blogs related to AIDS, patients, and their support networks, collected by Gopal [66] over a randomly selected three-day period in August 2005. A directed edge from one blog to another indicates that the former has a link to the latter in their web page (more specifically, the former refers to the latter in their so-called ‘blogroll’).

Note that both graphs are of roughly the same order (125 and 146 vertices, respectively), although the former has almost twice as many edges as the latter (300 versus 187). The lattice, however, is by definition highly uniform in its connectivity across vertices, whereas the blog network is not.

The simplest layout is a *circular layout*, wherein the vertices are arranged (usually equi-spaced) around the circumference of a circle. The edges are then drawn across the circle. Circular layouts of the lattice and blog networks are shown in Fig. 3.1.

```
#3.3 1 > igraph.options(vertex.size=3, vertex.label=NA,
2 +   edge.arrow.size=0.5)
3 > par(mfrow=c(1, 2))
4 > plot(g.l, layout=layout.circle)
5 > title("5x5x5 Lattice")
6 > plot(aidsblog, layout=layout.circle)
7 > title("Blog Network")
```

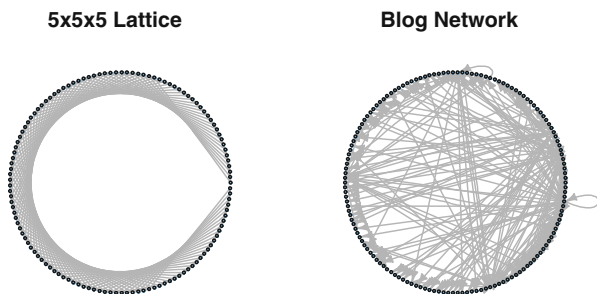


Fig. 3.1 Circular layouts

The visualization of the lattice is much more pleasing to the eye than that of the blog network, largely due to the low level of edge-crossings through the center of the circle. Ordering of the vertices around the circle is important with this type of layout—a random re-ordering of the vertices in the lattice, for example, would yield a picture much more like that of the blog network. Common vertex orderings for circular layouts include ordering by degree and grouping by common vertex attributes.

Often more effective for creating useful drawings are layouts based on exploiting analogies between the relational structure in graphs and the forces among elements in physical systems. One approach in this area, and the earliest proposed, is to introduce attractive and repulsive forces by associating vertices with balls and edges

with springs. If a literal system of balls connected by springs is disrupted, thereby stretching some of the springs and compressing others, upon being let go it will return to its natural state. So-called *spring-embedder* methods of graph drawing define a notion of force for each vertex in the graph depending, at the very least, on the positions of pairs of vertices and the distances between them, and seek to iteratively update the placement of vertices until a vector of net forces across vertices converges.

The method of Fruchterman and Reingold [60] is a commonly used example of this type. Applied to the lattice and blog networks,

```
#3.4 1 > plot(g.l, layout=layout.fruchterman.reingold)
      2 > title("5x5x5 Lattice")
      3 > plot(aidsblog, layout=layout.fruchterman.reingold)
      4 > title("Blog Network")
```

as shown in Fig. 3.2, we see that substantially more of the structure inherent to each network is now visible.

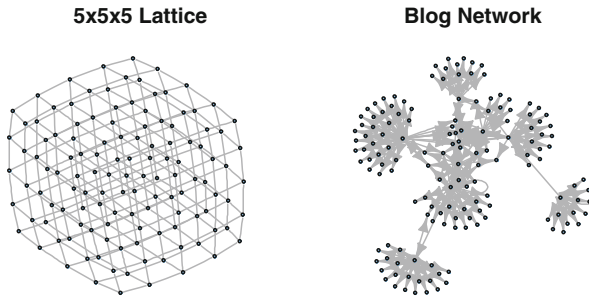


Fig. 3.2 Layouts using the method of Fruchterman and Reingold

Alternatively, motivated by the fact that it is possible to associate the collection of forces in spring systems with an overall system energy, another common approach to generating layouts is that of *energy-placement* methods. An energy, as a function of vertex positions, ostensibly is defined using expressions motivated by those found in physics. A vertex placement is chosen which minimizes the total system energy. A physical system with minimum energy is typically in its most relaxed state, and hence the assertion here is that a graph drawn according to similar principles should be visually appealing.

Methods based on multidimensional scaling (MDS), which have a long history in the social network literature, are of this type. The method of Kamada and Kawai [85] is a popular variant. Using this layout,

```
#3.5 1 > plot(g.l, layout=layout.kamada.kawai)
      2 > title("5x5x5 Lattice")
      3 > plot(aidsblog, layout=layout.kamada.kawai)
      4 > title("Blog Network")
```

the resulting visualizations of our lattice and blog networks are similar in spirit to those obtained using Fruchterman–Reingold. See Fig. 3.3.

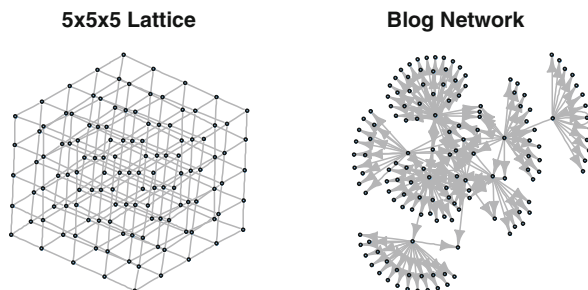


Fig. 3.3 Layouts using the method of Kamada and Kawai

In some cases, network graphs have special structure that it is desirable to accentuate. Trees are one such case. Consider, for example, the visualizations in Fig. 3.4.

```
#3.6 1 > g.tree <- graph.formula(1-+2,1-+3,1-+4,2-+5,2-+6,2-+7,
  2 +                               3-+8,3-+9,4-+10)
  3 > par(mfrow=c(1, 3))
  4 > igraph.options(vertex.size=30, edge.arrow.size=0.5,
  5 +   vertex.label=NULL)
  6 > plot(g.tree, layout=layout.circle)
  7 > plot(g.tree, layout=layout.reingold.tilford(g.tree,
  8 +   circular=T))
  9 > plot(g.tree, layout=layout.reingold.tilford)
```

With the circular layout, it is not obvious that the graph is a tree. However, with both the radial layout—in which edges radiate outward on concentric circles—and the layered layout, the structure of the graph is immediately apparent.

Similarly, bipartite graphs often are laid out with the two sets of vertices running across opposing rows (or down opposing columns)—one type of vertex within each

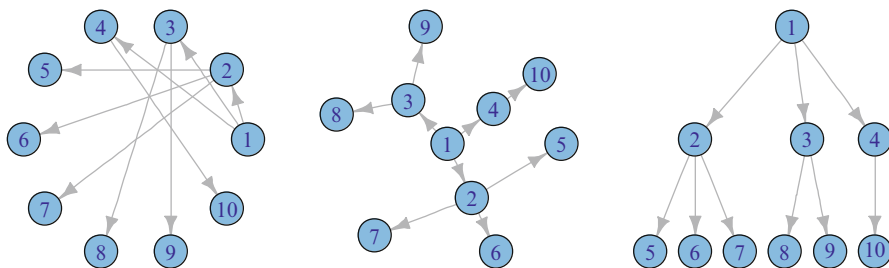


Fig. 3.4 Three layouts of the same tree: circular (*left*), radial (*center*), and layered (*right*)

row (column)—and with edges running between the rows (columns), in a manner reminiscent of railroad tracks. The visualization shown in Fig. 2.3, for example, is of this sort.

```
#3.7 1 > plot(g.bip, layout=-layout.bipartite(g.bip)[,2:1],
      2 +   vertex.size=30, vertex.shape=ifelse(V(g.bip)$type,
      3 +     "rectangle", "circle"),
      4 +   vertex.color=ifelse(V(g.bip)$type, "red", "cyan"))
```

3.4 Decorating Graph Layouts

While the relative positions of vertices and the placement of edges between them is clearly important in visualizing network graphs, additional network information—when available—can be incorporated into visualizations by varying characteristics like the size, shape, and color of vertices and edges. In particular, such techniques allow for the visualization of decorated graphs.

Consider, for example, the so-called ‘karate club network’ of Zachary [149]. Nodes represent members of a karate club observed by Zachary for roughly 2 years during the 1970s, and links connecting two nodes indicate social interactions between the two corresponding members. This dataset is somewhat unique in that Zachary had the curious fortune (from a scientific perspective) to witness the club split into two different clubs during his period of observation, due to a dispute between the head teacher and an administrator. Attribute information available for this network includes identification of the head teacher and the administrator, membership in one of the two factions underlying the eventual split, and relative frequency of interactions between members. While clearly more involved than just issuing a simple call to `plot()`, it is nevertheless straightforward to incorporate all of this information into a visualization of this network.

```
#3.8 1 > library(igraphdata)
      2 > data(karate)
      3 > # Reproducible layout
      4 > set.seed(42)
      5 > l <- layout.kamada.kawai(karate)
      6 > # Plot undecorated first.
      7 > igraph.options(vertex.size=10)
      8 > par(mfrow=c(1,1))
      9 > plot(karate, layout=l, vertex.label=V(karate))
     10 > # Now decorate, starting with labels.
     11 > V(karate)$label <- sub("Actor ", "", V(karate)$name)
     12 > # Two leaders get shapes different from club members.
     13 > V(karate)$shape <- "circle"
     14 > V(karate)[c("Mr Hi", "John A")]$shape <- "rectangle"
     15 > # Differentiate two factions by color.
     16 > V(karate)[Faction == 1]$color <- "red"
     17 > V(karate)[Faction == 2]$color <- "dodgerblue"
     18 > # Vertex area proportional to vertex strength
```

```

19 > # (i.e., total weight of incident edges).
20 > V(karate)$size <- 4*sqrt(graph.strength(karate))
21 > V(karate)$size2 <- V(karate)$size * .5
22 > # Weight edges by number of common activities
23 > E(karate)$width <- E(karate)$weight
24 > # Color edges by within/between faction.
25 > F1 <- V(karate)[Faction==1]
26 > F2 <- V(karate)[Faction==2]
27 > E(karate)[ F1 %--% F1 ]$color <- "pink"
28 > E(karate)[ F2 %--% F2 ]$color <- "lightblue"
29 > E(karate)[ F1 %--% F2 ]$color <- "yellow"
30 > # Offset vertex labels for smaller points (default=0).
31 > V(karate)$label.dist <-
32 +   ifelse(V(karate)$size >= 10, 0, 0.75)
33 > # Plot decorated graph, using same layout.
34 > plot(karate, layout=1)

```

The resulting visualization is shown in Fig. 3.5. Also shown, for comparison, is a visualization using the same layout coordinates (generated according to the Kamada–Kawai algorithm), but without any decoration. A substantial amount of additional information is communicated by way of the decorated visualization, where vertices are sized in proportion to their (weighted) degree, the relative frequency of interactions is shown using edge thickness, a change in vertex shape indicates the faction leaders, and colors are used to distinguish membership in the factions as well as edges joining within versus between the same factions.

A similar use of vertex color, shape, and size was used in the visualization of the Lazega lawyer network in Fig. 1.1. Note that within **igraph**, supplying the graphical parameters to the plotting command is an alternative to setting them as vertex and edge attributes. We demonstrate below.

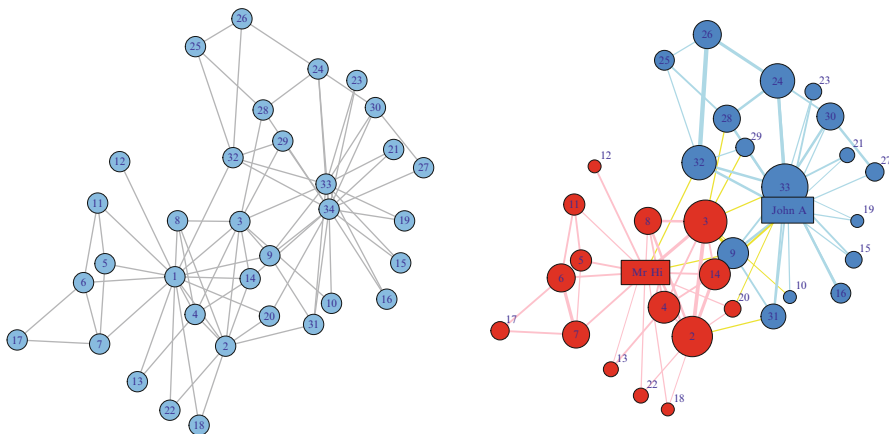


Fig. 3.5 Plain (*left*) and decorated (*right*) visualizations of Zachary's karate club network

```
#3.9 1 > library(sand)
2 > data(lazega)
3 > # Office location indicated by color.
4 > colbar <- c("red", "dodgerblue", "goldenrod")
5 > v.colors <- colbar[V(lazega)$Office]
6 > # Type of practice indicated by vertex shape.
7 > v.shapes <- c("circle", "square")[V(lazega)$Practice]
8 > # Vertex size proportional to years with firm.
9 > v.size <- 3.5*sqrt(V(lazega)$Years)
10 > # Label vertices according to seniority.
11 > v.label <- V(lazega)$Seniority
12 > # Reproducible layout.
13 > set.seed(42)
14 > l <- layout.fruchterman.reingold(lazega)
15 > plot(lazega, layout=l, vertex.color=v.colors,
16 +     vertex.shape=v.shapes, vertex.size=v.size,
17 +     vertex.label=v.label)
```

3.5 Visualizing Large Networks

Despite their sophistication, for all of the methods described so far, the graph drawings will tend to look increasingly cluttered as the number of vertices N_v nears 100 or so—and simply unintelligible for thousands of vertices or more—due to the finiteness of the available space and resolution.

For example, in Fig. 3.6 is shown a visualization of a subnetwork of French political blogs, extracted from a snapshot of over 1,100 such blogs on a single day in October of 2006 and classified by the “Observatoire Présidentielle” project as to political affiliation.² The network consists of 192 blogs linked by 1,431 edges, the latter indicating that at least one of the two blogs referenced the other.

```
#3.10 1 > library(sand)
2 > summary(fbblog)
3 IGRAPH UN-- 192 1431 --
4 attr: name (v/c), PolParty (v/c)
```

Nine political parties are represented among these blogs, and have been included as vertex attributes.

```
#3.11 1 > party.names <- sort(unique(V(fbblog)$PolParty))
2 > party.names
3 [1] " Cap21" " Commentateurs Analystes"
4 [3] " Les Verts" " liberaux"
5 [5] " Parti Radical de Gauche" " PCF - LCR"
6 [7] " PS" " UDF"
7 [9] " UMP"
```

² Original source: <http://observatoire-presidentielle.fr/>. The subnetwork used here is part of the **mixer** package in R. Note that the inherent directionality of blogs are ignored in these data, as the network graph is undirected.

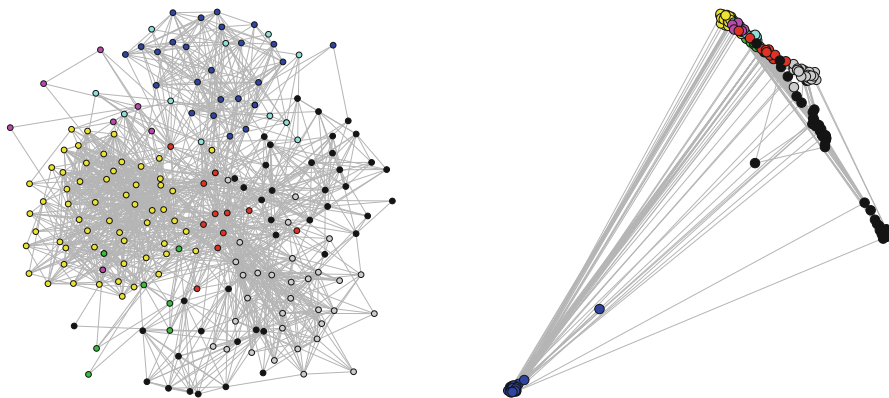


Fig. 3.6 Visualizations of the French political blog network using layouts generated by the Kamada–Kawai (*left*) and DrL (*right*) algorithms

The visualization shown on the left in Fig. 3.6 was produced using a layout generated by the standard Kamada–Kawai method.

```
#3.12 1 > set.seed(42)
      2 > l = layout.kamada.kawai(fblog)
      3 > party.nums.f <- as.factor(V(fblog)$PolParty)
      4 > party.nums <- as.numeric(party.nums.f)
      5 > plot(fblog, layout=l, vertex.label=NA,
      6 +     vertex.color=party.nums, vertex.size=3)
```

Note that, while it is indeed possible, with a bit of effort and the aid of color indicating political party affiliation, to distinguish some structure, the plot is nevertheless rather ‘busy’.

Fortunately, there are layout algorithms designed specifically for the purpose of visualizing large networks. For example, VxOrd [42], a visualization package produced by Sandia Labs, is an enhanced version of the spring-embedder methodology. It attempts to place vertices in clusters on the two-dimensional plane, with the help of sophisticated optimization methods to more efficiently search the space of possible graph drawings and a grid that helps reduce computation time from the typical $O(N_v^2)$ down to $O(N_v)$. In addition, it employs edge-cutting criteria, designed towards producing drawings that balance the detail with which both local and global structure are shown. VxOrd has been used by Boyack, Klavans, and Börner [21] to produce a map of the ‘backbone’ of Science, involving over 7,000 vertices (journals) and 16 million edges (co-citations). See also [91, Chap. 3.5.1] for a short summary of their analysis.

Another such example is the DrL method [106], which is based on VxOrd and is implemented in **igraph**. Applied to the French political blog data

```
#3.13 1 > set.seed(42)
      2 > l <- layout.drl(fblog)
      3 > plot(fblog, layout=l, vertex.size=5, vertex.label=NA,
      4 +     vertex.color=party.nums)
```

and shown on the right in Fig. 3.6, we find that the method has clustered blogs in a way that is strongly influenced by certain of the party affiliations, despite not having had this information.

When such clustering exists and may be characterized explicitly, either because it occurs with respect to a measured variable or because it is inferred through so-called graph partitioning methods,³ it may be useful to coarsen a network graph prior to visualization, replacing groups of vertices with single meta-vertices. This is demonstrated on the French political blog data in Fig. 3.7. Having first coarsened the network, by aggregating edges between groups,

```
#3.14 1 > fblog.c <- contract.vertices(fblog, party.nums)
      2 > E(fblog.c)$weight <- 1
      3 > fblog.c <- simplify(fblog.c)
```

we plot the resulting network.

```
#3.15 1 > party.size <- as.vector(table(V(fblog)$PolParty))
      2 > plot(fblog.c, vertex.size=5*sqrt(party.size),
      3 +     vertex.label=party.names,
      4 +     vertex.color=V(fblog.c),
      5 +     edge.width=sqrt(E(fblog.c)$weight),
      6 +     vertex.label.dist=1.5, edge.arrow.size=0)
```

In the resulting visualization, the size of the groups defined by political parties in the original network, and the numbers of edges between those groups, are reflected in vertex size and edge thickness, respectively. The relationships among political parties that was only suggested by the visualizations in Fig. 3.6 is now quite evident.

Alternatively, specific information we desire to be communicated through a network visualization might suggest that only a relevant subgraph(s) be shown. For example, sometimes it is useful to highlight the structure local to a given vertex, such as in the so-called *egocentric* network visualizations commonly used in the social network literature, which show the vertex, its immediate neighbors, and all edges among them.

As an illustration, consider again the karate network. Extracting the (first-order) neighborhoods surrounding each vertex

```
#3.16 1 > data(karate)
      2 > k.nbhds <- graph.neighborhood(karate, order=1)
```

we see, for example, that the neighborhoods pertaining to the head instructor (Mr Hi, vertex 1) and administrator (John A, vertex 34) are the largest.

```
#3.17 1 > sapply(k.nbhds, vcount)
      2 [1] 17 10 11 7 4 5 5 5 6 3 4 2 3 6 3 3 3
      3 [18] 3 3 4 3 3 3 6 4 4 3 5 4 5 5 7 13 18
```

³ See Chap. 4.4.

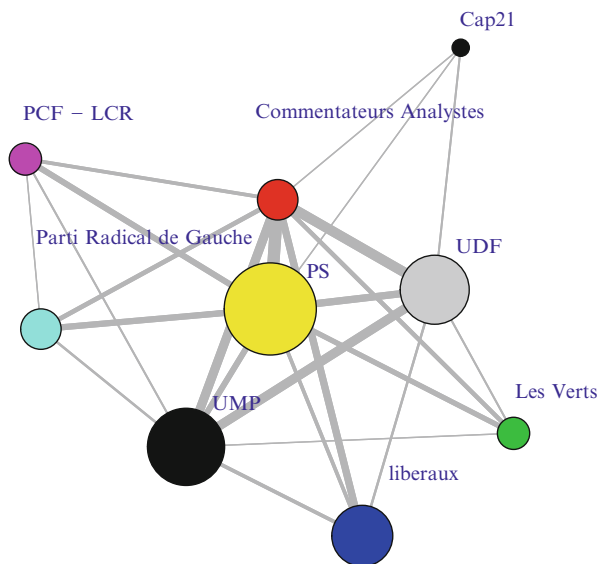


Fig. 3.7 Visualization of the French political blog network at the level of political parties

Pulling out these two largest subnetworks and plotting them,

```
#3.18 1 > k.1 <- k.nbhds[[1]]
      2 > k.34 <- k.nbhds[[34]]
      3 > par(mfrow=c(1,2))
      4 > plot(k.1, vertex.label=NA,
      5 +   vertex.color=c("red", rep("lightblue", 16)))
      6 > plot(k.34, vertex.label=NA,
      7 +   vertex.color=c(rep("lightblue", 17), "red"))
```

we obtain the visualizations shown in Fig. 3.8. Comparing these plots to that of the full karate network, as seen in Fig. 3.5, it is clear that these two subnetworks capture the vast majority of the structure in the full network.

3.6 Using Visualization Tools Outside of R

While using R for visualizing networks has the advantage of a programmatical interface, high quality graphics, and many graphics file formats, the network drawing itself is somewhat limited. For example, there are relatively few choices available for node and arrow shapes, and label placement is somewhat awkward, so that avoiding overlapping nodes and labels is not easy. Interactive editing of graph drawings is even more limited. However, if producing a network visualization of particularly high quality is a goal, there are certainly other software tools for this job. To close this chapter, we briefly touch upon some of these options, and discuss how data can be passed to them from R.

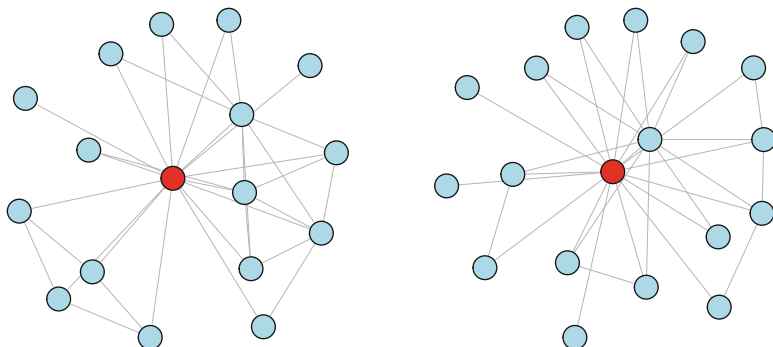


Fig. 3.8 Two ego-centric views of the karate club network of Fig. 3.5, from the perspectives of vertices 1 (*left*) and 34 (*right*), the authority figures (instructor and administrator, respectively, shown in *red*) about which the club ultimately split

Graphviz is a classic graph layout and drawing tool. It is a collection of standalone command line programs, with a separately developed interface to **R**, via the **Rgraphviz** package. An easy way to use an **igraph** graph with Graphviz is to write it to a file in the ‘dot’ file format that is supported by **igraph**’s `write.graph` function.

Pajek is one of the first network analysis and visualization tools. It is a standalone program for Microsoft Windows environments, and supports interactive visualization and editing as well. Pajek has its own file format, that is supported by **igraph**’s `write.graph` and `read.graph` functions (with some limitations), so transferring data between **R** and Pajek is usually easy.

Cytoscape is a modern cross-platform network analysis and visualization tool, written in Java, specializing in biological networks. It has several layout algorithms implemented and sophisticated interactive visualization, and it is under active development. Cytoscape can read and write the GML file format, also supported by **igraph**. In addition, it supports simple tables in text files, which is advantageous if the graph has many attributes.

Gephi is another standalone cross-platform tool for analysis and visualization of graphs, with sophisticated interactive editing. It has its own file format called ‘gexf’, and the **rgexf** package is able to convert ‘gexf’ to and from **igraph** graphs.

There are several other tools available, of course. These usually support some of the file formats **R** can read and write (e.g. GraphML, GML, CSV, etc.), so network data can be passed around easily. Visualization of vertex and edge attributes are sometimes more problematic, however.

3.7 Additional Reading

An extensive overview of principles and techniques for network graph visualization may be found in the edited volume of Kaufmann and Wagner [89]. For detailed development of many of the corresponding algorithms, see the text of di Battista, Eades, Tamassia, and Tollis [46]. Finally, for classical treatments of the more established body of work on the visualization of data in general, see the books by Tukey [140], Tufte [139], and Cleveland [31, 32].